

Reinforcement Learning Lab

Lesson 10: PPO

Alberto Castellini and Riccardo Fidanza

University of Verona

email: alberto.castellini@univr.it, riccardo.fidanza@studenti.univr.it

Academic Year 2025-26



UNIVERSITÀ
di VERONA
Dipartimento
di **INFORMATICA**

Environment Setup (CleanRL) - 1/3

We recommend using **Python 3.10** or **Python 3.11**. Create a virtual environment to isolate the project dependencies.

- **Conda (Recommended)**

```
conda create -n cleanrl python=3.11
conda activate cleanrl
```

- **venv (Unix / Linux / macOS)**

```
python3 -m venv .venv
source .venv/bin/activate
```

- **venv (Windows)**

```
python -m venv .venv
.\.venv\Scripts\activate
```

Note: If PowerShell blocks execution, run `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser` first.

Environment Setup (CleanRL) - 2/3

Once the environment is active, install the necessary dependencies.

- **Method A: Using requirements.txt (file in folder tools)**

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

- **Method B: Manual Installation (Alternative)**

If *requirements.txt* causes conflicts, you can manually install the core packages needed for CleanRL:

```
python -m pip install --upgrade pip  
pip install torch numpy gymnasium matplotlib tensorboard setuptools
```

- **Method C: Manual Installation (conda)**

(Recommended if you created the environment with Conda)

```
conda install pytorch numpy gymnasium matplotlib tensorboard setuptools -c pytorch
```

Notice: Remember to run these commands *inside* the activated `cleanrl` or `venv` environment.

Environment Setup (CleanRL) - 3/3

Update the repository of the lab:

```
cd RL-Lab
git stash
git pull
git stash pop
```

Training the RL Agent

You can train the agents using the provided scripts. These will automatically save the models and learning curves in the `runs/` folder.

- **Windows:** `.\lesson10_cleanRL_ppo_run_train.bat`
- **Unix / macOS:** `chmod +x lesson10_cleanRL_ppo_run_train.sh` then `./lesson10_cleanRL_ppo_run_train.sh`

Important Notes for the Lab:

- **Hyperparameters:** Scripts run with default values. You can override them manually from the terminal. Check the argument parsing section in the Python script to see all available options. Example:

```
python lesson10_cleanRL_ppo_train_code.py --env-id CartPole-v1 --total-timesteps 50
```

- **Batch Execution:** Inside the `.bat` or `.sh` files, you can modify the code to run a single environment or loop through a list of them.
- **Recommendation:** We strongly advise training **one environment at a time!** Each environment has different complexities and requires an ad-hoc set of tuned parameters.

CleanRL: Code Flow

Unlike Spinning Up, CleanRL is known for its **single-file implementation** approach.

The execution flow is structured as follows:

- **Execution Wrappers** (`.bat` or `.sh`): Scripts used to launch the training process, allowing you to easily run single or multiple environments in a row.
- **Training Script** (`lesson10_cleanRL_ppo_train_code.py`): A fully self-contained file managing the entire RL pipeline:
 - ▶ `parse_args()`: Defines and parses all the hyperparameters.
 - ▶ *Agent Class*: Contains the neural network definitions (both Actor and Critic) and the action-sampling logic.
 - ▶ *Main Loop*: Sets up vectorized environments, collects rollouts, computes Advantages, and performs the PPO network updates (**this is where you will implement the missing equations!**).
- **Testing Script** (`lesson10_cleanRL_test.py`): A lightweight script that reconstructs the *Agent*, loads the saved weights (`.cleanrl_model`), and runs a rendered episode to visually evaluate the trained policy.

The Skeleton Code

Inside `lesson10_cleanRL_ppo_train_code.py`, losses are temporarily initialized to zero to prevent crashes. Your task is to replace these placeholders with the actual PPO math:

```
# TODO: COMPUTE THE PPO POLICY LOSS
# pg_loss1 = ...
# pg_loss2 = ...
pg_loss = torch.tensor(0.0, device=device, requires_grad=True)

# Value loss
newvalue = newvalue.view(-1)
# TODO: COMPUTE THE VALUE LOSS
# if args.clip_vloss: ... (calculate clipped value loss)
v_loss = torch.tensor(0.0, device=device, requires_grad=True)
```

Hint: Use PyTorch operations like `torch.min`, `torch.clamp`, etc.

If stuck, check the CleanRL repo: <https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/ppo.py>

Algorithm 1 PPO-Clip

- 1: Input: initial policy parameters θ_0 , initial value function parameters ϕ_0
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
- 4: Compute rewards-to-go $\hat{R}_t$.
- 5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k} .
- 6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.

- 7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \left(V_{\phi}(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.

- 8: **end for**

PPO: summary of algorithm functioning

Question: How can we take the biggest possible improvement step on a policy using current data without stepping so far that we accidentally cause performance collapse?

- TRPO tries to solve this problem with a complex second-order method
- PPO is a first-order (and simpler) method that uses other tricks to keep new policies close to old

Main features of PPO

- PPO is an on-policy algorithm
- PPO works with discrete and continuous state spaces
- PPO works with discrete and continuous action spaces

In PPO, the policy parameters are updated according to

$$\theta_{k+1} = \arg \max_{\theta} \mathbb{E}[L(s, a, \theta_k, \theta)] \quad (1)$$

where

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s, a) \right) \quad (2)$$

$A^{\pi_{\theta_k}}(s, a)$ is the advantage function for the current policy, and ϵ is a (small) hyperparameter which restricts how far away the new policy is allowed to go from the old one.

PPO: Theory

In **CleanRL** (and standard practical implementations), a mathematically equivalent but simpler version of the objective function is considered:

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), g(\epsilon, A^{\pi_{\theta_k}}(s, a)) \right) \quad (3)$$

where

$$g(\epsilon, A) = \begin{cases} (1 + \epsilon)A & A \geq 0 \\ (1 - \epsilon)A & A < 0 \end{cases} \quad (4)$$

- **Positive advantage:** The objective will increase if the action becomes more likely. The *min* puts a limit to how much the objective can increase.
- **Negative advantage:** The objective will increase if the action becomes less likely. The *min* puts a limit to how much the objective can increase.

Clipping serves as a regularizer by removing incentives for the policy to change dramatically.

PPO: Implementation Flow

Unlike the old frameworks that used scattered files and separate functions, the update phase in **CleanRL** happens directly inside the main training loop.

- ➊ **Epochs & Minibatches:** The collected rollout data is flattened. The algorithm iterates for a set number of `update_epochs`, shuffling and processing data in minibatches.
- ➋ **Forward Pass:** For each minibatch, the *Agent* evaluates the observations to compute the new action probabilities (`newlogprob`), the policy entropy, and the state values (`newvalue`).
- ➌ **Combined Update:** Both the Actor (Policy) and the Critic (Value Function) are updated *together* in a single gradient descent step.
- ➍ **Loss Computation:** The optimizer minimizes a combined loss function:

$$\text{Loss} = \text{Policy Loss} - c_1 \times \text{Entropy} + c_2 \times \text{Value Loss}$$

PPO: Implementation - Policy Loss

- 1 Compute the probability ratio $r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}$.

Hint: For numerical stability, compute it using logarithms: $r_t(\theta) = e^{\ln(\pi_\theta(a|s)) - \ln(\pi_{\theta_{old}}(a|s))}$.
In PyTorch:

```
ratio = (newlogprob - b_logprobs[mb_inds]).exp()
```

- 2 Compute the unclipped surrogate objective (pg_loss1).

*Note: PyTorch optimizers **minimize** the loss. To maximize our PPO objective, we must negate it!*

```
pg_loss1 = -mb_advantages * ratio
```

- 3 Compute the clipped surrogate objective (pg_loss2) using `torch.clamp` with `args.clip_coef`.

- 4 The final policy loss (pg_loss) is the **maximum** between pg_loss1 and pg_loss2 (since we negated them), averaged over the minibatch using `torch.max()` and `.mean()`.

PPO: Implementation - Value Function Loss

- 5 The Value Function loss minimizes the error between the Critic's predictions (`newvalue`) and the actual empirical returns (`b_returns`).
- 6 **Standard MSE Loss:** Compute the squared difference:

$$L_{unclipped}^{VF} = (V_{\theta}(s) - R_t)^2$$

- 7 **Clipped Value Loss:** To prevent the value network from updating too aggressively, CleanRL clips the value function update:
 - ▶ Clip the `newvalue` around the old values (`b_values`) using `args.clip_coef` (via `torch.clamp`).
 - ▶ Calculate the clipped squared error: $L_{clipped}^{VF}$.
- 8 Take the **maximum** between the unclipped and clipped squared errors, compute the mean, and multiply by 0.5.

Gymnasium environments

Environment	States	Actions	Transition
MountainCar-v0	2 Continuous	3 Discrete	Deterministic
CartPole-v1	4 Continuous	2 Discrete	Deterministic
Acrobot-v1	6 Continuous	3 Discrete	Deterministic
LunarLander-v2	8 Continuous	4 Discrete	Deterministic
MountainCarContinuous-v0	2 Continuous	1 Continuous	Deterministic
Pendulum-v0	3 Continuous	1 Continuous	Deterministic
BipedalWalker-v3	24 Continuous	4 Continuous	Deterministic
FrozenLake-v0	16 Discrete	4 Discrete	Stochastic

How can we make PPO work on discrete state spaces?

RL Algorithm capabilities

Algorithm	States	Actions	Transition	On/Off-policy	V/Q	π
Policy gradient	D	D	Needed	Planning	V	No
MC Control	D	D	No	On-policy	Q	No
Sarsa	D	D	No	On-policy	Q	No
Q-Learning	D	D	No	Off-policy	Q	No
Dyna-Q	D	D	Learned	Off-policy	Q	No
DQN	D/C	D	No	Off-policy	Q(2)	No
REINFORCE	D/C	D/C	No	On-policy	No	Yes
REINFORCE+B	D/C	D/C	No	On-policy	V	Yes
A2C	D/C	D/C	No	On-policy	V	Yes
MCTS	D	D	Needed	Planning	Q	No
PPO	D/C	D/C	No	On-policy	V	Yes

Expected Results - Cart pole with PPO

Once the algorithm is complete it can be run with the command

```
lesson10_cleanRL_ppo_train_code.py --env=id CartPole-v1 --total-timesteps 50000
```

It produces textual output in the terminal and a chart with performance over training episodes

```
-----  
rollout/  
  ep_len_mean      200.14  
  ep_rew_mean      200.14  
time/  
  fps              1221  
  iterations        97/97  
  time_elapsed      40  
  total_timesteps   49664  
train/  
  entropy_loss      0.5696  
  learning_rate     0.000003  
  policy_loss       -0.0000  
  value_loss        61.0855  
-----
```

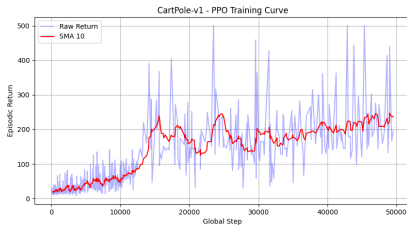


Figure: Mean return over episodes for CartPole-v1.

Expected Results

When the training concludes, check the newly created experiment folder inside the `runs/` directory (e.g., `runs/CartPole-v1_ppo_cleanRL_.../`). You will find:

- **TensorBoard Logs:** Event files containing all tracked metrics (losses, returns). Visualize them in your browser by running:

```
tensorboard --logdir runs/
```

- **Learning Curve (.png):** A pre-generated plot showing the episodic return over the training steps (raw and smoothed).
- **Model Weights (.cleanrl_model):** A PyTorch state dictionary containing the trained Agent's weights. To visualize your trained agent playing the game, use the test scripts:

```
.\run_test.bat      # Windows  
./run_test.sh      # Unix / macOS
```

Training on other environments

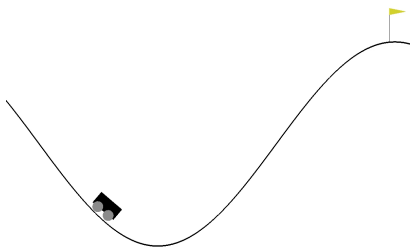
Compare the PPO agent's performance across different environments:

- **Change Environment:** Edit the `env` argument in your `run_train.bat` or `run_train.sh` scripts.
- **Action Spaces:** Some are discrete, some continuous. How does the code select the correct policy output?
(Hint: Check the `Agent` class and `is_continuous` flag in `ppo_cleanRL.py`).
- **Stuck?** Check the `README.md` for recommended hyperparameters!

Questions for you

- Does this network architecture work with *discrete state spaces* (e.g., `FrozenLake`)? What must change?
- Can you run the algorithm on other Gymnasium environments?

Environment: MountainCar-v0



- **Goal:** Train an underpowered car to reach the top of a hill by rocking back and forth to build momentum.
- **State:** 2 continuous values: *Position* and *Velocity*.
- **Actions (Discrete):**
 0. Accelerate left
 1. Don't accelerate
 2. Accelerate right
- **Challenge:** The reward signal is notoriously difficult. Standard PPO will fail if you don't tweak the hyperparameters! (See *README.md*).